Лабораторная работа №18

Получение данных из удаленной СУБД через API

О чем лабораторная?

В прошлой работе приложение научилось регистрировать пользователя, выполнять вход и сохранять JWT-токен. Теперь используем этот токен при получении данных с сервера и свяжем ответ API с экраном приложения.

В примере проекта будем загружать данные текущего пользователя и его объявления. В своем проекте можно выбрать другой набор данных, например список категорий, избранные объявления или историю сообщений.

Главное

- приложение отправляет запрос API, а сервер сам обращается к удаленной базе данных
- защищенный запрос должен содержать заголовок `Authorization: Bearer ...`
- ответ сервера нужно преобразовать в Java-модели
- состояние интерфейса надо обновлять только после получения ответа

Часть 1. Подготовка и выбор данных

Откройте документацию API:

`http://95.31.156.238:8000/docs`

Изучите операции:

- `GET /user/me` - получить данные авторизованного пользователя
- `GET /user/{user_id}/items` - получить объявления конкретного пользователя
- `GET /items` - получить страницу объявлений с поиском и фильтром
- `GET /categories` - получить массив категорий объявлений

В текущем проекте уже есть модели и методы для `GET /user/me` и `GET /items`. Идентификатор пользователя в API можно получить из ответа `GET /user/me`.

Задание

Сделаем загрузку объявлений пользователя на экран "Мои объявления"

Часть 2. Проверьте модели ответа

Ответ `GET /items` содержит объект с массивом `items` и информацией о странице:

```
{
  "items": [
```

```
    {
      "id": 10,
      "title": "Гитара",
      "description": "Акустическая гитара",
      "price": 7000,
      "deal_type": "sale",
      "photo_url": null,
      "category": {
        "id": 2,
        "name": "Музыка",
        "emoji": "🎸"
      },
      "author": {
        "id": "UUID",
        "username": "student",
        "about_me": null
      }
    }
  ],
  "meta": {
    "total": 1,
    "page": 1,
    "page_size": 20,
    "total_pages": 1
  }
}
```

Проверьте, что в проекте есть следующие классы:

- `ItemResponse`
- `AuthorResponse`
- `CategoryResponse`
- `MetaResponse`
- `PaginatedResponse<T>`

Названия полей должны соответствовать JSON. Например, для поля `deal_type` в Java можно оставить имя `deal_type`.

Если серверное DTO и модель интерфейса отличаются, не смешивайте их. Например, `ItemResponse` описывает ответ API, а `Item` хранит данные в формате, удобном для карточки приложения.

Часть 3. Проверьте авторизацию запроса

Откройте класс, который создает `Retrofit` и `OkHttp-клиент`. В нем должен быть `authInterceptor`, добавляющий токен:

```
Interceptor authInterceptor = chain -> {
    String token = tokenStorage.getAccessToken();

    Request.Builder request = chain.request().newBuilder();
```

```
    if (token != null && !token.isEmpty()) {
        request.header("Authorization", "Bearer " + token);
    }

    return chain.proceed(request.build());
};
```

Убедитесь, что `authInterceptor` подключен к `OkHttpClient`, как и наш старый логгирующий:

```
OkHttpClient client = new OkHttpClient.Builder()
        .addInterceptor(logging)
        .addInterceptor(authInterceptor)
        .build();
```

Часть 4. Загрузите текущего пользователя

Добавьте в `ApiService` метод, если его еще нет:

```
@GET("user/me")
Call<UserResponse> getMe();
```

В нужной Activity выполните запрос:

```
ApiService api = RetrofitClient.getApiService(getApplicationContext());

api.getMe().enqueue(new Callback<UserResponse>() {
    @Override
    public void onResponse(
            Call<UserResponse> call,
            Response<UserResponse> response
    ) {
        if (response.isSuccessful() && response.body() != null) {
            UserResponse user = response.body();
            usernameTextView.setText(user.getUsername());
        } else if (response.code() == 401) {
            // токен недействителен. Нужно вернуть пользователя на экран входа
        } else {
            // покажите понятное сообщение об ошибке
        }
    }

    @Override
    public void onFailure(Call<UserResponse> call, Throwable t) {
        // нет соединения или сервер недоступен.
    }
});
```

Не оставляйте в профиле только имя из `SharedPreferences`. **Локальные данные могут устареть.**

Часть 5. Загрузите объявления пользователя

В API есть отдельный метод для объявлений пользователя:

`GET /user/{user_id}/items`

Сначала запросите `GET /user/me`, получите UUID пользователя, затем передайте его в запрос объявлений.

Методы в интерфейсе API:

```
@GET("items")
Call<PaginatedResponse<ItemResponse>> getItems(
        @Query("page") int page,
        @Query("page_size") int pageSize,
        @Query("search") String search,
        @Query("category_id") Integer categoryId
);

@GET("user/{user_id}/items")
Call<PaginatedResponse<ItemResponse>> getUserItems(
        @Path("user_id") String userId,
        @Query("page") int page,
        @Query("page_size") int pageSize
);
```

Пример загрузки в `MyItemsActivity`:

```
ApiService api = RetrofitClient.getApiService(getApplicationContext());

api.getMe().enqueue(new Callback<UserResponse>() {
    @Override
    public void onResponse(Call<UserResponse> call, Response<UserResponse>
userResponse) {
        if (!userResponse.isSuccessful() || userResponse.body() == null) {
            showMessage("Не удалось получить профиль пользователя");
            return;
        }

        String userId = userResponse.body().getId();
        api.getUserItems(userId, 1, 20).enqueue(
                new Callback<PaginatedResponse<ItemResponse>>() {
            @Override
            public void onResponse(
                    Call<PaginatedResponse<ItemResponse>> call,
                    Response<PaginatedResponse<ItemResponse>> response
            ) {
                if (!response.isSuccessful() || response.body() == null) {
                    showMessage("Сервер не вернул объявления");
                    return;
                }
```

```
                List<ItemResponse> myItems = response.body().getItems();

                if (myItems.isEmpty()) {
                    showMessage("У вас пока нет объявлений");
                } else {
                    recyclerView.setAdapter(
                            new ItemAdapter(MyItemsActivity.this, myItems)
                    );
                }
            }

            @Override
            public void onFailure(
                    Call<PaginatedResponse<ItemResponse>> call,
                    Throwable t
            ) {
                showMessage("Проверьте подключение к интернету");
            }
                });
    }

    @Override
    public void onFailure(Call<UserResponse> call, Throwable t) {
        showMessage("Проверьте подключение к интернету");
    }
});
```

Часть 6. Обновите интерфейс в трех состояниях

Экран должен показывать разные состояния:

1. идет загрузка
2. данные успешно загружены
3. произошла ошибка или список пуст

Добавьте в разметку индикатор загрузки, например `ProgressBar`:

```
<ProgressBar
    android:id="@+id/progress_bar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center" />
```

Перед запросом покажите индикатор:

```
progressBar.setVisibility(View.VISIBLE);
```

В `onResponse()` и `onFailure()` скройте его:

```
progressBar.setVisibility(View.GONE);
```

Не оставляйте пустой экран без объяснения. Пользователь должен понимать, загружаются ли данные, список пуст или возникла ошибка.

Часть 7. Самостоятельная работа

Загрузите объявления текущего пользователя на экран "Мои объявления"

1. Получите UUID пользователя через `GET /user/me`
2. Выполните запрос `GET /user/{user_id}/items`
3. Покажите полученные объявления в существующем `RecyclerView`
4. Если объявлений нет, покажите сообщение "У вас пока нет объявлений"
5. Если произошла ошибка сети, покажите сообщение с предложением повторить загрузку

Требования

- UUID пользователя получается через `GET /user/me`
- объявления загружаются через `GET /user/{user_id}/items`
- полученные объявления отображаются в `RecyclerView`
- при пустом списке показывается понятное сообщение

Дополнительно (по желанию)

1. Сохраните последний успешный ответ локально и покажите его при отсутствии сети